# How the DOM Diff Works

BrickPHP renders both sides of a change on the server and ships the difference. This document follows one interaction end to end: how a component becomes a tree of DOM nodes, how that tree becomes HTML, and how two of those trees are compared into a list of patch operations the browser can apply.

---

## 1 The idea in one paragraph

Most component frameworks diff in the browser: the client keeps a virtual DOM, re-renders it after a state change, and compares new against old. BrickPHP keeps nothing in the browser. When an event fires, the server renders the tree **twice in the same request** — once for the state as it was before the event (`OLD`), once for the state after the handler ran (`NEW`) — compares those two server-side trees, and returns a flat list of patch operations. The browser's only job is to apply them.

> **THE CONSEQUENCE THAT SHAPES EVERYTHING ELSE**
>
> The server never looks at the real DOM. It infers what the browser is showing by re-deriving it from persisted state. So OLD must be an exact reproduction of what is on screen — which is why the framework carries a state hash, replays the previous URL, and falls back to a full reload the moment those assumptions break (§9).

---

## 2 The request lifecycle

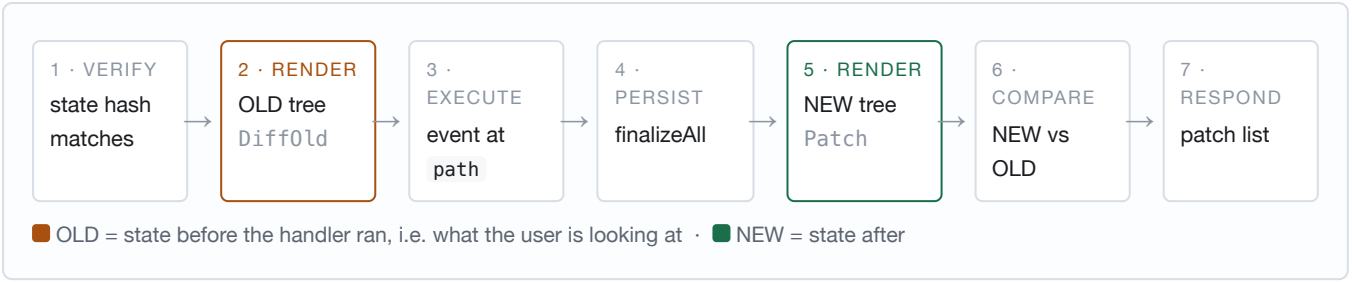
### 2.1 First load — a GET

The entry `App` renders once in `RenderPhase::Initial`, state is persisted, every event listener in the tree is bound, and a full HTML document goes out. A hash of the state is embedded as `window.__BRICK_HASH`; the client echoes it back on every subsequent POST.

```
// Brick::handleGet — src/Brick.php:960
$ui = $entry->render($state, null, RenderPhase::Initial);
$entry->finalize($state);          // persist component state
$ui->bindEvents();                 // queue client-side listener wiring
$stateHash = self::computeStateHash($state);   // sha1(serialize(all state))
```

### 2.2 Every interaction after that — a POST

The client posts `{event, path, value, bindings, hash, previousPath}`. The server then runs the three-render dance that produces the diff:




```
// Brick::handleEvent — src/Brick.php:800–841 (abridged)
$oldUi = $oldApp->render($state, null, RenderPhase::DiffOld);   // 2

$node = $oldUi->findByPath($path);                               // 3
$node?->executeEvent($event, $state, $value);

Component::finalizeAll($state);                                 // 4

$newUi = $newApp->render($state, null, RenderPhase::Patch);     // 5

Component::processDeleted();                                    // lifecycle

$patcher = new Patcher();                                       // 6
$newUi->compare($oldUi, $patcher);
```

Three details in that sequence are easy to miss and matter a great deal:

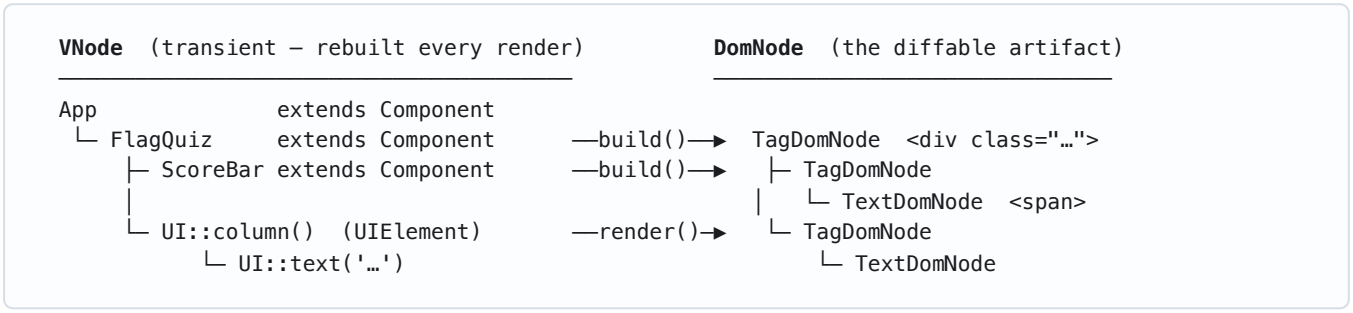
- **The event runs against the OLD tree.** `findByPath($path)` locates the node the user actually clicked in the tree as it was rendered — the handler closure lives on that node.
- **The OLD render is done at `previousPath`, not the current URL.** After a `popstate` the browser URL has already moved on; without the override, OLD would render the new route too, the diff would come out empty, and the stale DOM would never be patched.
- **`finalizeAll` sweeps every component, not just the event's owner.** A handler can capture `$this` from an ancestor and mutate its state; finalizing only the owner would silently drop those writes.

### 2.3 The three render phases

| RENDERPHASE | WHEN | WHAT IT MEANS FOR A COMPONENT |
|---|---|---|
| `Initial` | GET, full page | Whole tree is new — `created()` fires for every component. |
| `DiffOld` | POST, first render | Replays an existing tree. No `created()`, no `rendered()` — this phase must not cause side effects. |
| `Patch` | POST, second render | `created()` only where the path is newly occupied; otherwise `updated()`. `shouldRender()` is consulted here and nowhere else. |

## 3 How a component is built

There are two trees, and keeping them apart is the key to reading the codebase. The **VNode tree** — components and fluent UI builders — is scaffolding that exists only during a render. It produces the **DomNode tree**, and that is the artifact that gets serialised to HTML and diffed. Nothing above `DomNode` ever takes part in the compare.




### 3.1 The build → render contract

Every component implements one abstract method, `build(): VNode`, returning the subtree for its position. The framework calls `render()`, which wraps `build()` in identity resolution, state restoration and lifecycle:

```
// Component::render — src/VNode/Component.php:280 (abridged, in order)
$this->ensureInitialized();                        // useState() registrations
$savedState = $resolved->getState($pathKey);       // restore from the StateManager
$this->setState($savedState);
$this->stateSnapshot = $this->captureStateSnapshot();   // serialize()d props+state

// register under path:class so OLD and NEW instances can be matched up
$phase === RenderPhase::Patch
    ? self::$newRegistry[$pathKey] = $this
    : self::$oldRegistry[$pathKey] = $this;

$firstAppearance ? $this->created() : $this->updated();

// memo hook: false here skips build() AND the diff for this whole subtree
if ($phase === RenderPhase::Patch && !$this->shouldRender($oldInstance)) {
    return new NoOpDomNode();
}

$rendered = $this->build()->render($state, $this, $phase);

if ($parent === null) {
    $rendered->assignPaths();       // one O(n) pass over the whole tree
}
```

### 3.2 Component identity: `path:class`

A component is identified by `getStateKey()` — its path joined by dots, a colon, and its concrete class name (e.g. `0.1.2:Samples\FlagQuiz\Components\ScoreBar`). That single string does a lot of work:

- It is the storage key for the component's state in the `StateManager`.

- It matches the OLD instance to the NEW instance so `shouldRender($old)` has something to compare against.
- `array_diff_key($oldRegistry, $newRegistry)` is literally how `deleted()` is computed — a component present in OLD but absent from NEW is gone, and its bound state is dropped from storage.

> **IMPLICATION**
> Component identity is *positional*. Move a component to a different index and the framework sees the old one deleted and a new one created — its state resets. This is the same trade-off that makes `key()` matter for lists (§6.4).

### 3.3 The DomNode family

| CLASS | RENDERS AS | ROLE IN THE DIFF |
|---|---|---|
| `TagDomNode` | `<tag …>children</tag>` | The workhorse. Attributes, classes, events, children. |
| `TextDomNode` | `<span>text</span>` | Text, always wrapped in an element so it owns a stable path. |
| `ListDomNode` | `<tag>` + keyed children | Keyed collection diffed with a Levenshtein alignment (§6.5). |
| `NoOpDomNode` | empty string | `compare()` does nothing — the browser keeps whatever it had. |
| `RawContent` | content verbatim | Never diffed; escape hatch for pre-rendered markup. |

## 4 Paths: the addressing scheme

A patch is useless without a way to say *where*. BrickPHP addresses every node by its positional path from the root — an array of child indices, `[0, 3, 1]` meaning "root → child 0 → child 3 → child 1".

```
// DomNode::assignPaths — src/UI/DomNode.php:94
public function assignPaths(array $parentPath = []): static
{
    $this->path = $parentPath;
    $this->managed = true;
    $this->assignChildPaths();      // child i gets [...$this->path, $i]
    return $this;
}
```

Assignment happens exactly once per render, in a single pass from the true root (`$parent === null`). Doing it per-node would make path assignment $O(n^2)$ — every node re-walking its own subtree.

The same path array is used for four different things, which is what keeps the system coherent:

1. **Patch targets** — every operation carries the path of the node it acts on.
2. **Event addressing** — the client posts the path of the element that was clicked; the server finds the handler with `findByPath()`.
3. **Listener binding** — each `EventRegistration` is added or removed at a path.
4. **Component identity** — the `path:class` state key of §3.2.

> **WHY TEXT IS WRAPPED IN A SPAN**
>
> The client resolves paths by walking `element.children`, which skips text nodes. If bare text nodes were allowed into the tree, server-side indices (which counted them) and client-side indices (which didn't) would drift apart, and every patch after the first text node would land on the wrong element. Wrapping text in a `<span>` keeps a single, unambiguous indexing scheme on both sides.

---

## 5 How the HTML is built

Serialisation is a plain recursive `toHtml()` over the DomNode tree. It runs in two situations: for the whole tree on the initial GET, and for individual subtrees that a patch needs to insert or replace. In the second case it happens *after* the diff, when the collected operations are handed out:

```php
// Patcher::getOperations — src/VNode/Patcher.php:150
return array_map(function ($op) {
    if (isset($op['node']) && $op['node'] instanceof DomNode) {
        $op['html'] = $op['node']->toHtml();     // serialise only what's actually sent
        unset($op['node']);
    }
    return $op;
}, $this->operations);
```

That ordering is deliberate: a node that the diff decides is unchanged is never serialised at all.

### What lands in the markup

- **Classes** are self-describing utility names, merged from the node's own class list and any explicit `class` attribute, then de-duplicated. The client decodes unfamiliar names into real CSS rules as patched HTML arrives — so no stylesheet payload rides along with the patches.
- **No inline `on*` handlers.** Every listener is wired imperatively through its `EventRegistration`, driven by the diff (§8).
- **Wireframe metadata** (`data-wf-label`, `data-wf-file`, `data-wf-line`) is emitted straight into the attribute string and deliberately kept out of the node's `$attributes` array — so the diff never sees it and dev-mode annotations can never generate patches.
- **Void elements** (`img`, `input`, `br`, …) are looked up in a keyed constant and closed without a child list.

---

## 6 The compare

Everything below hangs off one abstract method:

```php
// src/UI/DomNode.php:183
abstract public function compare(DomNode $other, Patcher $patcher): void;
```

> **READ THIS BEFORE READING THE SOURCE**
>
> The call is `$newUi->compare($oldUi, $patcher)`. Inside every `compare()` implementation, `$this` is the **NEW** node and `$other` is the **OLD** one. Reversing them in your head inverts the meaning of every insert and delete.

### 6.1 The order of checks in `TagDomNode::compare`

src/UI/TagDomNode.php:551

```
// 1. Opted out entirely — don't even walk the subtree.
if ($this->frozen) { return; }

// 2. Different node type or different tag → wholesale replacement.
if (!$other instanceof TagDomNode || $this->tag !== $other->tag) {
    $other->unbindEvents();
    $patcher->replaceNode($this->path, $this);
    $this->bindEvents();
    return;
}

// 3. A whole-node invalidation cascades to every child.
if ($this->invalidated) {
    foreach ($this->children as $child) { $child->setInvalidated(true); }
}

// 4. Attributes: present-and-different, or forced → set. Vanished → remove.
foreach ($thisAttrs as $name => $value) {
    $forced = $this->invalidated || isset($this->invalidatedAttrs[$name]);
    if ($forced || !isset($otherAttrs[$name]) || $otherAttrs[$name] !== $value) {
        $patcher->setAttribute($this->path, $name, $value);
    }
}
foreach ($otherAttrs as $name => $value) {
    if (!isset($thisAttrs[$name])) { $patcher->removeAttribute($this->path, $name); }
}

// 5. Classes are compared as a whole and re-emitted as one attribute.
if ($classForced || $thisClasses !== $otherClasses) {
    $patcher->setAttribute($this->path, 'class', $this->serializeClasses($thisClasses));
}

// 6. This element survived, so diff its listeners rather than rebinding them.
$this->diffEvents($other);

// 7. Recurse.
$this->compareChildren($other, $patcher);
```

Note step 2: a replacement is *structural*, not incremental. The whole subtree is serialised and shipped, so a changing tag name is the most expensive thing the diff can encounter. Text is cheaper — `TextDomNode::compare` emits a single `replace_text` when the content differs, and falls back to `replace_node` only if the OLD node wasn't text at all.

### 6.2 Choosing a child strategy

Children are compared positionally unless *both* the old and new child lists are fully keyed — every single child carrying a non-null key:

```php
// TagDomNode::compareChildren — src/UI/TagDomNode.php:630
$newKeyed = $this->childrenAreKeyed();
$oldKeyed = $other->childrenAreKeyed();

($newKeyed && $oldKeyed)
    ? $this->compareChildrenKeyed($other, $patcher)
    : $this->compareChildrenPositional($other, $patcher);
```

One unkeyed child in a list of a hundred silently downgrades the whole list to positional comparison.

### 6.3 Positional comparison

Index *i* in NEW is compared against index *i* in OLD. Surplus old children are deleted **in reverse order** — deleting forwards would shift the indices of everything still to be removed.

```
  OLD   [ A ][ B ][ C ][ D ]        NEW   [ A ][ X ][ C ]

        i=3  delete_node [.,3]      ← reverse order first
        i=0  A vs A → recurse, no change
        i=1  B vs X → recurse, patches
        i=2  C vs C → recurse, no change
```

### 6.4 Keyed comparison

`src/UI/TagDomNode.php:683`

Each key maps to a FIFO queue of the old indices that carry it. Walking the new children in order, the algorithm accepts the next candidate for a key only if its old index is **strictly greater** than the last index it matched; otherwise the candidate is discarded and it tries the next one. A new child that finds no usable candidate becomes a fresh insertion.

```php
$oldQueues = [];
foreach ($other->children as $i => $child) { $oldQueues[$child->getKey()][] = $i; }

$lastMatched = -1;
foreach ($this->children as $newIdx => $newChild) {
    $key = $newChild->getKey();
    // discard candidates that would require moving backwards
    while (!empty($oldQueues[$key]) && $oldQueues[$key][0] <= $lastMatched) {
        array_shift($oldQueues[$key]);
    }
    if (empty($oldQueues[$key])) { $matched[$newIdx] = null; continue; }   // → insert
    $oldIdx = array_shift($oldQueues[$key]);
    $matched[$newIdx] = $oldIdx;
    $usedOld[$oldIdx] = true;
    $lastMatched = $oldIdx;
}
```

The strictly-increasing rule is what lets the diff produce correct DOM order *without ever emitting a move operation*. Anything that would have to travel backwards is expressed as a delete plus an insert instead.

> **THE TRADE-OFF, STATED PLAINLY**
>
> There are no move patches. Reordering a list — dragging a row upward, reversing a sort — is realised as removal and re-insertion, so any DOM state living in those elements (focus, scroll position, an in-progress text selection, CSS transitions) is lost. Appends, removals and in-place edits are handled optimally; reordering is correct but not cheap.

### 6.5 `ListDomNode` and the Levenshtein alignment

`ListDomNode` takes a different route. It reduces both child lists to arrays of keys and runs a modified Levenshtein edit-distance over them, then walks the resulting DP table backwards to recover the edit script.

```
// Levenshtein::populate — src/Dom/Levenshtein.php:19
$neighbours = [
    [$dp[$i-1][$j-1][0] + ($same ? 0 : 1), ($same ? self::SKIP : self::SUBSTITUTE)],
    [$dp[$i-1][$j][0],                      self::DELETE],
    [$dp[$i][$j-1][0],                      self::INSERT],
];
$min = array_reduce(
    $neighbours,
    fn($acc, $item) => $item[0] < $acc[0] ? $item : $acc,
    $neighbours[0],
);
$dp[$i][$j] = [$min[0] + ($same ? 0 : 1), $min[1]];
```

The four action codes map directly onto patch operations:

| ACTION | MEANING | WHAT `LISTDOMNODE::COMPARE` DOES |
|---|---|---|
| `SKIP` | Keys match | Recurse into the pair — `$newItem->node->compare($oldItem->node, …)` |
| `SUBSTITUTE` | Key changed at this slot | `update_at` ; unbind old listeners, bind new |
| `INSERT` | Present only in NEW | `insert_at` ; bind the new subtree's listeners |
| `DELETE` | Present only in OLD | `remove_at` at an offset-adjusted index; unbind |

The edit script comes back reversed (the traversal runs from the bottom-right of the table toward the origin), so it is `array_reverse` d before being applied. Two counters run alongside: `$newIndex` advances on everything except a delete, and `$removeOffset` compensates for indices already removed.

> **A HARD LIMIT WORTH KNOWING**
>
> The backtrace loop in `Levenshtein::diff` is bounded by a literal `$count = 20` . Since recovering the script takes roughly `max(count(OLD), count(NEW))` steps, a `ListDomNode` holding more than about twenty children stops emitting operations before it reaches the start of the list — the earliest items produce no patches at all. `TagDomNode` 's keyed path (§6.4) has no such bound, which makes it the safer choice for long lists today.

## 7 The patch protocol

The diff's entire output is a JSON array of operations. Nine types exist; each carries a `path` , and the ones that add markup carry pre-serialised `html` .

| TYPE | PAYLOAD | CLIENT ACTION ASSETS/BRICK.JS:108 |
|---|---|---|
| `replace_node` | path, html | Clean up listeners, then `replaceWith` the parsed element |
| `insert_node` | path, html | Insert at the last path segment within the parent |
| `delete_node` | path | Clean up listeners, then `remove()` |
| `replace_text` | path, text | Assign `textContent` |
| `set_attribute` | path, name, value | `setAttribute`, plus property sync for `value` / `checked` |
| `remove_attribute` | path, name | `removeAttribute` |
| `insert_at` | path, index, html | Insert before `parent.children[index]`, or append |
| `remove_at` | path, index | Clean up listeners and remove that child |
| `update_at` | path, index, html | Clean up listeners and replace that child |

Resolution on the client is the mirror image of `assignPaths`:

```
// assets/brick.js:91
function findNodeByPath(path, includeTextNodes = false) {
    let node = document.body.firstElementChild;      // the root element
    for (let i = 0; i < path.length; i++) {
        if (!node) return null;
        node = includeTextNodes ? node.childNodes[path[i]] : node.children[path[i]];
    }
    return node;
}
```

> **ORDER IS LOAD-BEARING**
>
> Patches are applied strictly in the order the server emitted them, and each one is resolved against the DOM *as mutated by its predecessors*. This is why deletions are emitted in reverse index order and why the list algorithms carry offset counters — the server is reasoning about a moving target.

## 8 Event binding rides the diff

There is no separate pass to reconcile listeners. Registration and de-registration are emitted by `compare()` itself, at exactly the moments a node's participation in the tree changes:

| SITUATION | LISTENER HANDLING |
|---|---|
| Node replaced (type/tag change) | `$other->unbindEvents()` then `$this->bindEvents()` — whole subtrees |
| Child inserted | `bindEvents()` on the new subtree |
| Child deleted | `unbindEvents()` on the departing subtree |
| Node survives | `diffEvents($other)` — bind only listeners that appeared, unbind only those that vanished |

```php
// TagDomNode::diffEvents — src/UI/TagDomNode.php:258
foreach ($this->events as $event => $data) {
    if (!isset($old->events[$event])) { $data['registration']->add($this->path); }
}
foreach ($old->events as $event => $data) {
    if (!isset($this->events[$event])) { $data['registration']->remove($this->path); }
}
```

An unchanged listener on a surviving node is left strictly alone — it is neither rebound nor re-emitted, which is what keeps a re-render from disturbing in-flight interactions.

---

## 9 Controlling and constraining the diff

### 9.1 The four escape hatches

| MECHANISM | SET WITH | EFFECT |
|---|---|---|
| `frozen` | `setFrozen()` | Diff bails at this node. No attributes, text or children compared; the subtree isn't even rendered during a POST. For genuinely static UI. |
| `invalidated` | `invalidate()` | Force-emit patches regardless of equality, cascading to every descendant. For values the server-vs-server diff structurally cannot see as changed. |
| `invalidatedAttrs` | `invalidateAttr($n)` | The same, narrowed to one attribute; everything else diffs normally. |
| `shouldRender` | override on a component | Returning `false` substitutes a `NoOpDomNode`, skipping both `build()` and the diff for that subtree. |

> **WHY `INVALIDATE()` HAS TO EXIST**
> OLD and NEW are rendered milliseconds apart in the same request, so anything derived from the clock, from randomness, or from an external source produces the *same* value on both sides. The diff correctly concludes "no change" — while the browser is still showing a value from a render minutes ago. A clock's hands are the canonical case: they must be force-patched because the server has no way to notice they moved.

> **WHY `SHOULDRENDER()` DEFAULTS TO `TRUE`**
> Returning `false` from a parent also skips every descendant's `build()`. A child's own state can change without touching the parent's, and the parent would never see it — so a `false` default would be unsound. Memoization is safe to opt into on leaves (`hasChanged($old)` compares serialized state snapshots taken before any handler ran), and needs care above them.

### 9.2 Guarding the assumption that OLD is accurate

The whole design rests on the server being able to reproduce the user's screen. Three mechanisms defend that:

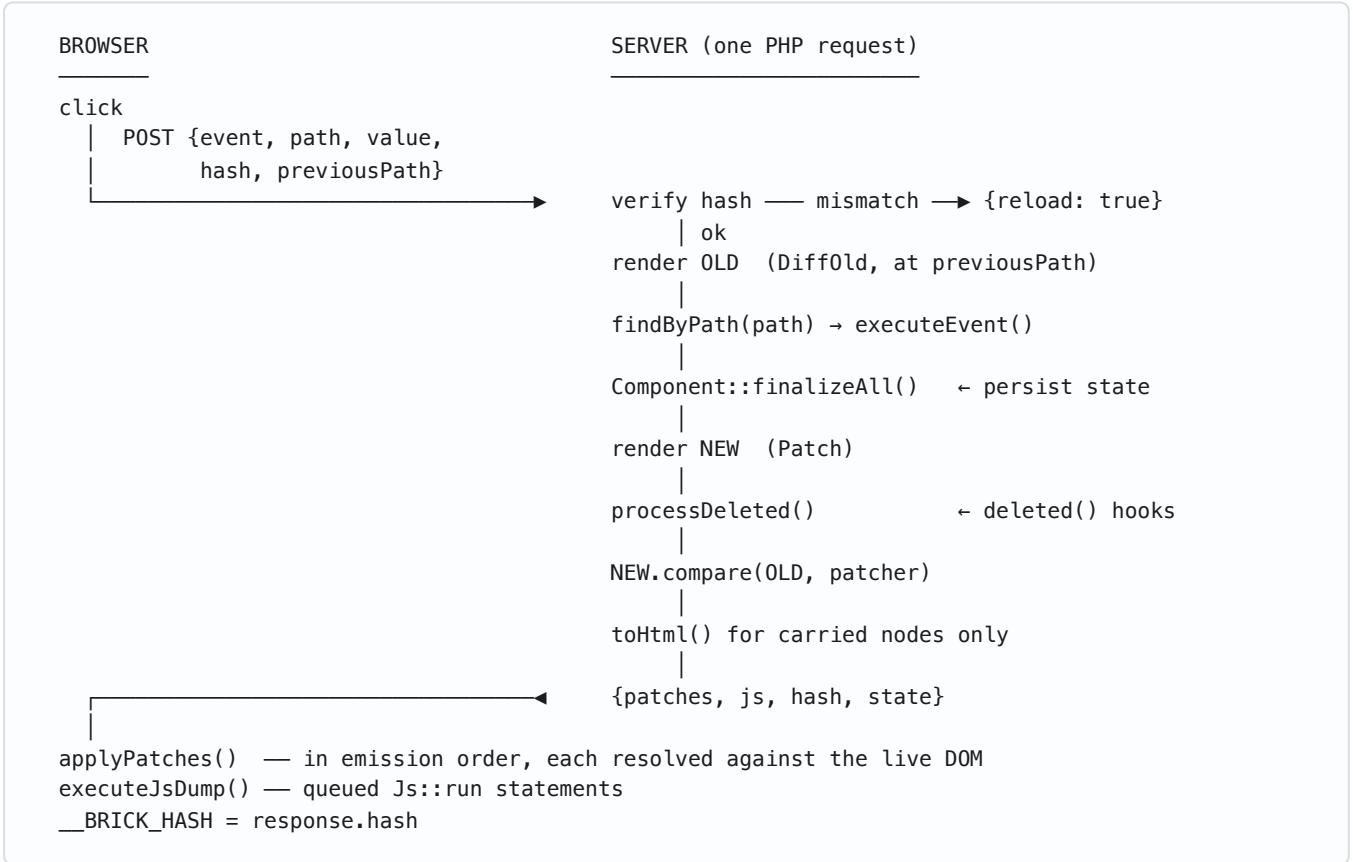
- **The state hash.** The client echoes the `sha1(serialize(all state))` it was rendered against. If the server's current state hashes differently — another tab mutated it, a deploy reshaped it — the response is `{success: false, reload: true}` rather than a patch list computed against a tree the user isn't looking at.

- `previousPath`. The URL the DOM was last rendered for, used to render OLD after the browser has already moved the address bar (§2.2).
- **Blanket recovery.** If restoring serialized state throws during either render — typically because the persisted shape no longer matches the current code — all state is cleared and the client is told to reload into a clean default.

### 9.3 Cost model

| OPERATION | COST |
|---|---|
| Text change | One `replace_text`. Cheapest possible patch. |
| Attribute or class change | One `set_attribute` per changed attribute; classes go as a single merged string. |
| Tag or node-type change | Whole subtree serialised and shipped. Most expensive. |
| Append / remove in a keyed list | One operation per affected item. |
| Reorder in a keyed list | Delete + insert for everything that moves backwards (§6.4). |
| Unchanged subtree | Traversed, compared, never serialised — `toHtml()` runs only for nodes a patch actually carries. |

## 10 The whole cycle, at a glance



## 11 Source map

| FILE | RESPONSIBILITY |
|---|---|
| `src/Brick.php:735–870` | The POST cycle: payload parsing, hash check, OLD render, event execution, NEW render, the diff call, response assembly |
| `src/Brick.php:960+` | The GET cycle: initial render, listener binding, hash embedding |
| `src/VNode/Component.php:280` | `render()` — state restoration, registries, lifecycle, memo hook, path assignment |
| `src/VNode/BaseComponent.php:91` | The stateless render flow and the `shouldRender` contract |
| `src/VNode/RenderPhase.php` | The three phases |
| `src/VNode/Patcher.php` | Operation collection; `toHtml()` at hand-off |
| `src/UI/DomNode.php` | Base node: paths, keys, frozen/invalidated flags, the `compare()` contract |
| `src/UI/TagDomNode.php:551–726` | The main comparison, plus positional and keyed child strategies |
| `src/UI/ListDomNode.php:127` | Levenshtein-driven keyed list comparison |
| `src/Dom/Levenshtein.php` | DP table and edit-script recovery |
| `assets/brick.js:91–215` | Path resolution and patch application |

BrickPHP internals · generated from the source at ~/projects/brickphp · line references are accurate as of the commit this was written against.